

Measuring abstraction in brains and neural networks

Sep 10, 2026

Abstraction is one of the defining features of intelligence. We can define it as the ability to extract a general property of an object from different examples of it. This definition encompasses why abstraction is so important: it is one of the principles that underpins the ability to generalise.

If you look outside the window and see droplets of water falling from the sky, you know that it's raining. This is true regardless of where you happen to be, the time of day, and whether the rain is heavy or light. Rain is rain, and because you understand that, if you go on holiday to a place you've never been and it starts raining, you will wish you had brought your umbrella (or had chosen a sunnier destination!) You don't need to learn what rain looks like in that particular place to understand this. In other words, you can abstract the concept away from the sensory particularities.

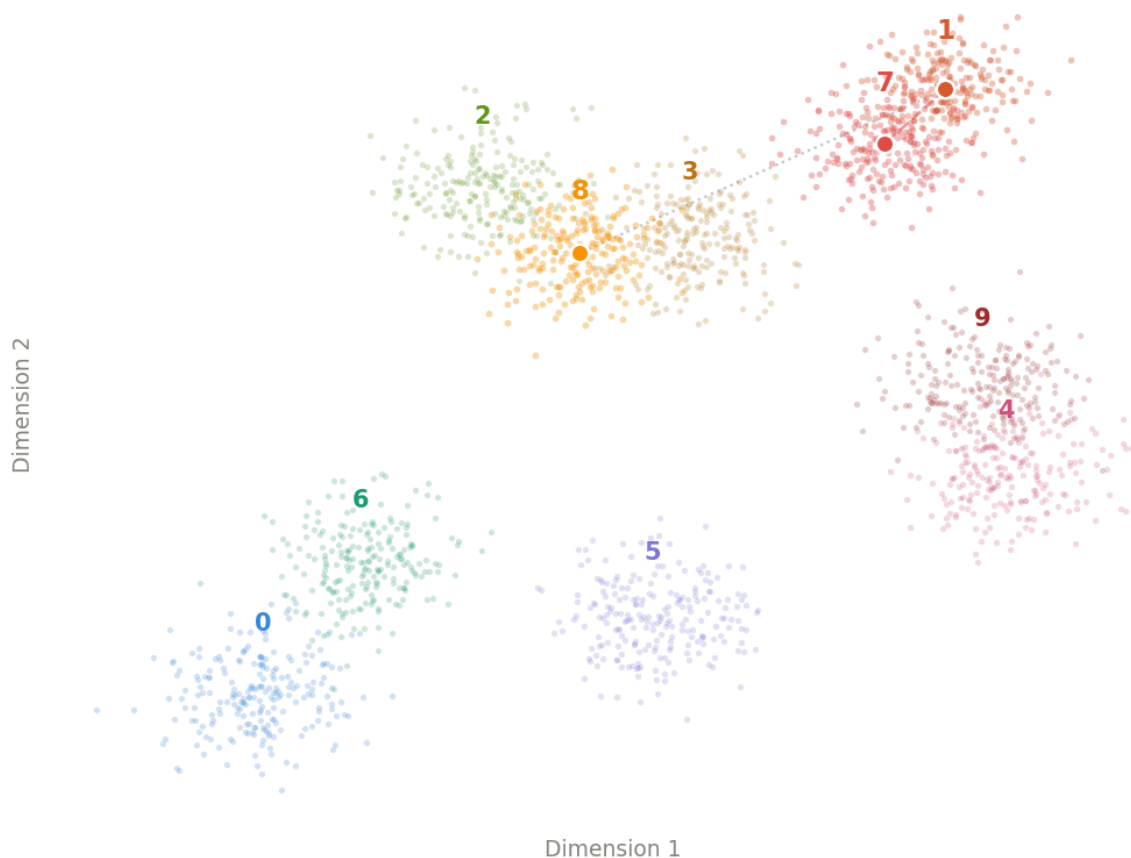
This kind of abstraction is also a desirable property of a representation in artificial neural networks (ANNs), and is the very thing we're trying to get them to learn by training them on lots of examples. But what makes a neural representation abstract? The answer is invariance. We want a representation which is the same regardless of the irrelevant particularities of a given example. We could conceive that somewhere in the cortex, a rain neuron (or subspace) exists whose activity is the same whether you're at home or on your drizzly holiday¹.

Now we have an idea about the type of representation we need, how can we actually measure it? That question is the focus of this [2020 paper](#) from the labs of Stefano Fusi and C. Daniel Salzman entitled “The Geometry of Abstraction in the Hippocampus and Prefrontal Cortex”.

Representational geometry

To ask questions about representations, we first need to define the representation space. Typically, whether we're looking at high-dimensional neural recordings we think of its representation in terms of its firing rate², and the activations in an ANN as the analogue of this.

If we define a space where each dimension is the firing rate of a neuron or unit, then we can plot the state of the network as a point in that rate space. Then, we can start to compare the state of the network in different conditions in this space.



Examples of reduced population responses to images of digits. Each cluster corresponds to a different digit, illustrating how representational similarity in firing-rate space can reflect shared visual features.

Distance in this space gives us a natural measure of representational similarity which we can use to begin to question what the network encodes. Imagine a dataset where we've got the responses of a network (or some biological neurons) to images of handwritten digits, like the MNIST dataset. If a particular cortical area or network layer encodes shape features, we might expect its activations for 1 to be nearer those for 7 than, say, 8, since 1 and 7 share a dominant vertical stroke. to be nearer then share a dominant vertical stroke. Beyond simple distances, we might draw the vectors connecting cluster centres, or fit the separating hyperplane of a linear decoder trained to discriminate between two conditions. These geometric objects – distances, angles between vectors, orientations of decision boundaries – are the basic vocabulary of representational geometry. It is a flexible framework for

quantifying the information encoded in a brain or neural network. And with a carefully designed experiment, it lets you ask precise questions about abstraction.

Designing an experiment for abstraction

To ask questions about abstraction over a feature of interest, we need it to vary independently of other features. This permits us to split the data into groups of two – a process called “dichotomisation”, which I’ll dive into soon – such that each group differs by only one of these features, but contains trials for all values of the other features. We can achieve this with considered experimental design. Bernardi *et al* designed a behavioural experiment with monkeys which gave them 8 different conditions that enabled them to dissociate representations of context, value and action from neural recordings.

The experiment went like this: the monkeys had to fixate on a screen and hold down a button, and would then be presented with 1 of 4 images on the screen. Depending on the image, the correct response would either be to continue holding the button or to release it. Half of these images would be followed by a reward for the correct response, but the other half would not. The monkeys would be trained to perform this task and then at a random trial the stimulus-response-outcome mappings would change to a new ruleset, defining two different “contexts”.

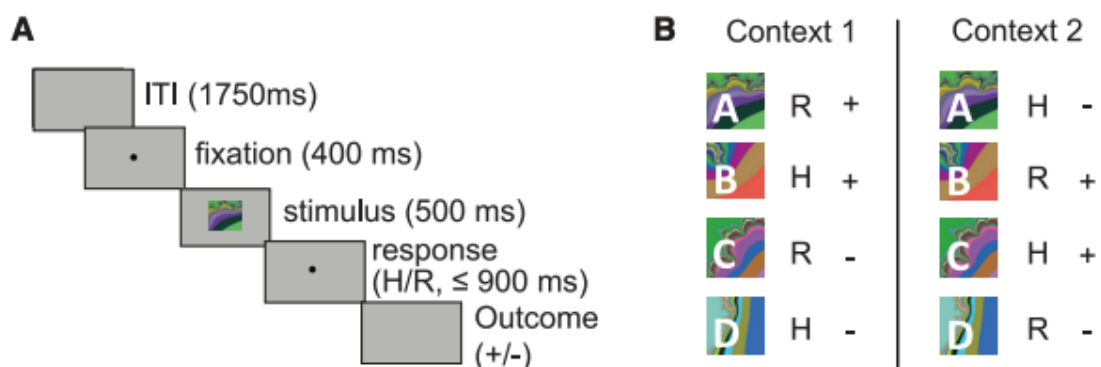


Fig. 1 from Bernardi et al. 2020. Panel B columns show each image's correct response (R = release, H = hold) and reward outcome (+/-) in each context.

Notice that from these 8 conditions we can create a dichotomy for action (hold/release), value (correct response rewarded/unrewarded) and context. But how can we actually measure if these features are encoded abstractly? To answer that, we need a metric called CCGP.

Cross-condition generalisation performance

Cross-condition generalisation performance (CCGP) is a decoding-based metric that, with the right conditions, can be used to measure abstraction. It is one of the key contributions of the Bernardi et al paper.

It starts with dichotomisation. Now we have the 8 conditions outlined above, we can split them up into groups of two. Having built these dichotomies, we can then train a linear classifier to discriminate the conditions on one side of the dichotomy from the conditions on the other. With 8 conditions, there are 35 possible dichotomies (

, since A vs B is the same dichotomy as B vs A), including 3 “special” dichotomies which have a clear interpretation: context (contexts 1 and 2 on opposite sides), value (rewarded/unrewarded on opposite sides), and action (release and hold on opposite sides).

To measure the presence of one of these variables, we could train a decoder on its dichotomy. A common approach is to use cross-validation to train multiple decoders and measure the average accuracy, as is often done in systems neuroscience studies. Whilst this would let us measure the presence of this variable, and cross-validation controls for generalisation across trials, it doesn't tell us anything about generalisation across specific conditions (and therefore abstraction). For a representation to be abstract, for example of

value, it has to be encoded the same in context 1 as in context 2. This is where CCGP comes in.

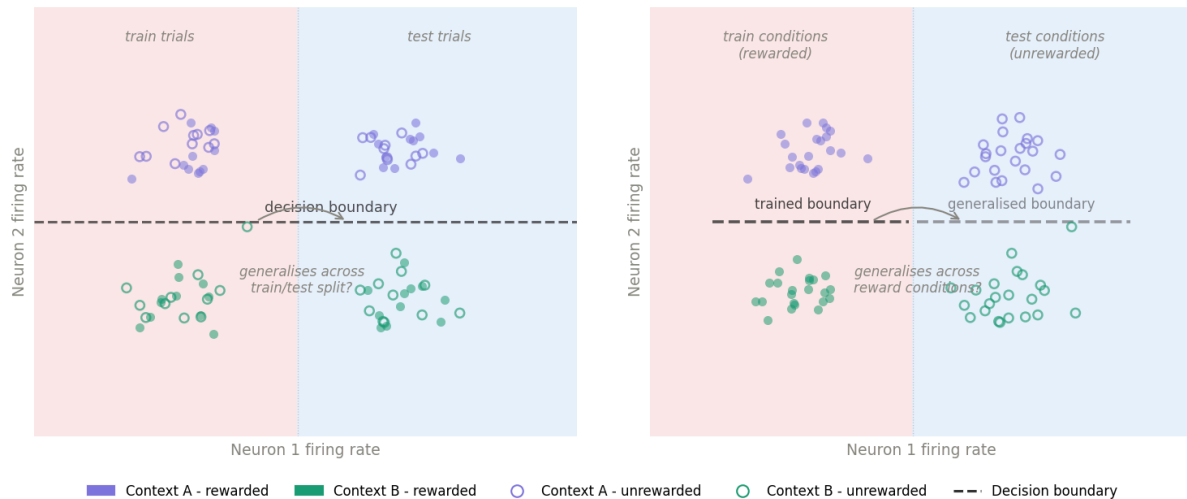


Illustration of the difference between standard cross-validated decoding (left) and CCGP (right). In standard decoding all conditions contribute to both train and test sets; in CCGP the classifier is trained on one subset of conditions and tested on a held-out subset it has never seen.

CCGP's key difference from regular cross-validated decoding accuracy is that the conditions in the training and test sets are constructed so that samples from different conditions are not mixed. For example, for context CCGP we're decoding context 1 against context 2, and we have both high and low value trials on either side of the dichotomy. Rather than choosing them randomly, CCGP would train on the high value trials and test on the low value trials. Since the decoder trained on high value trials has never seen the low value trials, this is like testing the decoder in a novel situation. If the decoder can generalise across them then it indicates that the feature of interest is encoded in an abstract form. For context CCGP, we could also train on the stimulus A trials and test on the stimulus B trials. In the paper, they compute CCGP over all permutations of train-test pairs (but to emphasise, train/test trials are grouped by condition) and take the average to obtain the final CCGP for a dichotomy.

Parallelism score

When you think about why a decoder might generalise, it is because the axis separating two conditions of interest is shared across contexts. Another way to measure this, without training any decoders, is to draw a line between the centres of the clusters encoding a pair of conditions on either side of a dichotomy and within an interpretable group (e.g. the line connecting the high-value trials for context 1 and context 2). We could also draw a line between the low-value trials for context 1 and 2. If the context representation is abstract, then these two coding vectors should align, and we can check this by measuring the angle between them. This is essentially what the parallelism score does, except it considers all possible coding vectors for a given dichotomy and takes the maximum over all pairings.

We can formalise this as follows. For a given dichotomy, the conditions are split into a “positive” group and a “negative” group (each containing 4 conditions for this dataset). A permutation simultaneously matches each condition in with a unique condition in, yielding 4 pairs and therefore 4 coding vectors. We compute the normalised coding vector for each pair as

where

is the mean firing rate vector (across neurons) for the i -th condition in the positive group and μ_j is the mean firing rate vector for its matched condition in the negative group. The denominator normalises to a unit vector so we only care about direction, not magnitude.

Once we have these 4 coding vectors, we average the cosine of the angle between every unique pair:

The factor of 6 comes from the number of unique pairs among 4 vectors ($\binom{4}{2} = 6$)

). Finally, we take the maximum over all possible pairings, since we don't want to assume a priori which pairing best reflects the underlying geometry:

While CCGP asks whether a decoder can generalise across conditions, PS asks why. A high PS tells us directly that the coding geometry is consistent across conditions, providing the geometric substrate that makes generalisation possible. They are complementary, and if both are high, we have converging evidence for a genuinely abstract representation.

Putting it to work

Armed with these two metrics, we can now begin to ask questions about the nature of coding in neural recordings or neural network activations. In their paper, Bernardi et al. used this to compare representations in three brain regions: ACC, DLPFC and HPC. In summary, they found that while almost all dichotomies were decodable everywhere, very few had high CCGP. Context and value dichotomies had the highest CCGP and PS in all 3 areas, while action CCGP was highest in ACC and DLPFC but notably not in HPC. The authors explain that, if you were to simply analyse clustering in firing rate space, you might incorrectly conclude that HPC encodes action in an abstract manner, highlighting the utility of the CCGP metric.

In summary, these metrics provide a beautifully simple framework for quantifying abstraction in a way which is applicable to both neural data and neural network models. I have found it useful in my own work comparing representations in RNNs and calcium imaging recordings, and the paper has been cited over 650 times as of August 2026, which speaks to how influential it has been. Recently, I have been exploring how it can be used to measure abstraction within LLMs. Stay tuned for a post on this.

Thanks for reading :)

-
1. In reality, the representation may be compositional - for example there may be separate subspaces representing weather and water, and the concept of rain read out as their conjunction. The point about abstraction still applies. ↩
 2. There are many other ways for neurons to encode information, but I'm focusing purely on rate coding here. ↩